

ALGORITHM ARENA

Iterated Prisoner's Dilemma

Match Protocol and Tournament Evaluation

DOCUMENT B: ARENA AND EVALUATION SPECIFICATION

Problem Identifier:	axelrod-ipd
Specification Level:	Document B (Evaluation and Match Protocol)
Tournament Mode:	Round-Robin
Match Length (v1):	Fixed Horizon ($T = 200$)
Version:	1.0.0
Date:	August 2026

This specification defines the language-neutral controller protocol, fixed match execution rules, round-robin tournament mechanics, reproducible RNG seeding hierarchy, and baseline evaluation suite for Axelrod IPD.

Contents

1	Overview and Architecture	1
2	Language-Neutral Controller Protocol	1
2.1	Abstract Controller Interface	1
2.2	Observation and Action Schemas	1
2.3	Language Adapters	1
2.3.1	Python 3 Adapter	1
2.3.2	C++ Adapter (C++17 / C++20)	2
2.3.3	Rust Adapter	2
2.3.4	Java Adapter (Java 17 / 21 LTS)	2
2.3.5	Go Adapter (Go 1.22+)	2
3	Match Protocol	3
3.1	Fixed Match Horizon (v1 Standard)	3
3.2	Step Execution Sequence	3
3.3	Fallback and Error Handling	3
4	Randomness and Deterministic Reproducibility	3
4.1	Seed Hierarchy	4
4.2	The Determinism Guarantee	4
5	Tournament Format and Leaderboard	4
5.1	Round-Robin Tournament	4
5.2	Primary Ranking Metric	4
5.3	Secondary Metrics	4
5.4	Tie-Breaking Order	5
6	Benchmark Baseline Opponents	5

1 Overview and Architecture

This document (Document B) specifies how user algorithm submissions are structured, instantiated, executed, and benchmarked on the Algorithm Arena competitive platform for the **Iterated Prisoner’s Dilemma (Axelrod IPD)** problem.

The evaluation architecture enforces strict separation between:

1. **Mathematical Stage Game (Document A):** The invariant problem logic ($T = 5, R = 3, P = 1, S = 0$, history h_t).
2. **Evaluation Protocol (Document B - This Document):** Controller interfaces, deterministic match execution, baseline suites, and leaderboard rankings.
3. **Execution Sandbox (Document C):** Hardware resource bounds, process isolation, and security constraints.

2 Language-Neutral Controller Protocol

2.1 Abstract Controller Interface

A competitive submission is formalized as an autonomous, stateful decision-making controller π . The controller interacts with the Arena Arbiter via a standardized lifecycle protocol:

Lifecycle Method	Signature / Arguments	Semantic Behavior
<code>initialize()</code>	$\emptyset \rightarrow \emptyset$	Invoked once when the process is spawned to allocate memory and load static weights.
<code>reset()</code>	$\emptyset \rightarrow \emptyset$	Invoked prior to each pairwise match. Must reset all internal state, buffers, and opponent models.
<code>step()</code>	Observation $\rightarrow$ Action	Invoked synchronously at each round $t \in [0, 199]$ to output the next action.

2.2 Observation and Action Schemas

- **Input Observation:**

$$o_t = \left(t, \mathbf{a}_{0:t-1}^{(\text{self})}, \mathbf{a}_{0:t-1}^{(\text{opp})} \right)$$

where $t \in \mathbb{N}_0$ is the 0-indexed round number, $\mathbf{a}_{0:t-1}^{(\text{self})} \in \{0, 1\}^t$ is the list of own past actions in the active match, and $\mathbf{a}_{0:t-1}^{(\text{opp})} \in \{0, 1\}^t$ is the list of opponent past actions. At $t = 0$, both history lists are empty.

- **Output Action:** An integer $a_t \in \{0, 1\}$ representing:

$$a_t = 1 \implies \text{Cooperate } (C), \quad a_t = 0 \implies \text{Defect } (D)$$

2.3 Language Adapters

2.3.1 Python 3 Adapter

```

1 class BaseStrategy:
2     """Standard Python Adapter for Algorithm Arena: Axelrod IPD."""
3
4     def __init__(self, name: str = "MyStrategy"):
5         self.name = name
6
7     def reset(self) -> None:
8         """Resets all internal match memory and state."""
9         pass
10
11     def step(self, history_self: list[int], history_opp: list[int]) -> int:
12         """Returns 1 (Cooperate) or 0 (Defect)."""
13         raise NotImplementedError

```

2.3.2 C++ Adapter (C++17 / C++20)

```

1 #include <vector>
2 #include <string>
3
4 class IStrategy {
5 public:
6     virtual ~IStrategy() = default;
7     virtual void reset() = 0;
8     virtual int step(const std::vector<int>& history_self,
9                     const std::vector<int>& history_opp) = 0;
10 };

```

2.3.3 Rust Adapter

```

1 pub trait Strategy {
2     fn reset(&mut self);
3     fn step(&mut self, history_self: &[u8], history_opp: &[u8]) -> u8;
4 }

```

2.3.4 Java Adapter (Java 17 / 21 LTS)

```

1 package arena.axelrod;
2
3 import java.util.List;
4
5 public interface Strategy {
6     /**
7      * Resets internal match state and memory buffers.
8      */
9     void reset();
10
11     /**
12      * Computes the next action (1 = Cooperate, 0 = Defect).
13      */
14     int step(List<Integer> historySelf, List<Integer> historyOpp);
15 }

```

2.3.5 Go Adapter (Go 1.22+)

```

1 package axelrod
2
3 // Strategy defines the interface for Axelrod tournament controllers.
4 type Strategy interface {
5     // Reset clears all internal state between pairwise matches.
6     Reset()
7
8     // Step computes the action for the current round (1 = Cooperate, 0 = Defect).
9     Step(historySelf []int, historyOpp []int) int
10 }

```

3 Match Protocol

3.1 Fixed Match Horizon (v1 Standard)

For the MVP and v1 baseline specification:

Match Duration: Each pairwise match executes for exactly $T = 200$ rounds.

Note: Variable-horizon and noisy game variations are reserved for subsequent specification versions (e.g., axelrod-ipd-v2).

3.2 Step Execution Sequence

For each round $t = 0, 1, \dots, 199$:

1. Arena Arbiter sends read-only snapshots of $(\mathbf{a}_{0:t-1}^{(1)}, \mathbf{a}_{0:t-1}^{(2)})$ to Agent 1 and Agent 2 simultaneously.
2. Both agents compute their actions $a_t^{(1)}$ and $a_t^{(2)}$ within the step timeout limit (10 ms).
3. The Arbiter records $(a_t^{(1)}, a_t^{(2)})$, computes stage payoffs $(r_t^{(1)}, r_t^{(2)})$ using the payoff matrix, and updates cumulative scores.
4. Actions are committed to the public match history.

3.3 Fallback and Error Handling

- **Invalid Return Value:** If an agent returns a value other than 0 or 1, the Arbiter overrides the decision to $D = 0$ (Defect) for that round and records a penalty warning.
- **Step Timeout:** If an agent fails to respond within 10 ms, the action defaults to $D = 0$.
- **Process Crash / Fatal Exception:** The crashing agent forfeits the match, receiving 0 points for all remaining rounds, while the opponent receives $T = 5$ points per remaining round.

4 Randomness and Deterministic Reproducibility

To guarantee competitive integrity, dispute auditability, and instant match replayability, Algorithm Arena enforces a **Hierarchical PRNG Seeding Protocol**:

4.1 Seed Hierarchy

$$\text{Seed}_{\text{tournament}} \xrightarrow{\text{derive}} \text{Seed}_{\text{match}} \xrightarrow{\text{derive}} (\text{Seed}_{\text{controller 1}}, \text{Seed}_{\text{controller 2}})$$

1. **Match Seed Derivation:** Each pairwise match (i, j) is assigned a unique 64-bit integer seed computed deterministically via cryptographic hashing:

$$\text{Seed}_{i,j} = \text{HMAC-SHA256}(\text{TournamentSecret}, \text{concat}(i, j, \text{match_index}))$$

2. **Controller Seed Injection:** Prior to calling `reset()`, the platform seeds the standard pseudo-random number generator (PRNG) of the controller sandbox (e.g., `Python random.seed()`, `numpy.random.seed()`).

4.2 The Determinism Guarantee

Invariant: Given an identical submission binary/code and an identical $\text{Seed}_{\text{match}}$, the controller's sequence of decisions across all 200 rounds must be **100% bitwise deterministic and reproducible**.

Matches with non-deterministic unseeded execution or asynchronous race conditions are automatically flagged during verification.

5 Tournament Format and Leaderboard

5.1 Round-Robin Tournament

Let $\mathcal{P} = \{\pi_1, \pi_2, \dots, \pi_M\}$ be the pool of verified strategies.

- Every strategy π_i plays a pairwise match of $T = 200$ rounds against every strategy π_j (for $j \neq i$).
- Each strategy also plays one self-play match between two independent instances (π_i, π_i) initialized with distinct controller seeds.

5.2 Primary Ranking Metric

Strategies are ranked on the global leaderboard by their **Average Payoff Per Round**:

$$\bar{u}_i = \frac{1}{M \cdot T} \sum_{j=1}^M S_{i,j}$$

where $S_{i,j}$ is the total score accumulated by strategy π_i in its 200-round match against π_j .

5.3 Secondary Metrics

1. **Win-Loss-Tie Record:** Number of head-to-head match victories $(S_{i,j} > S_{j,i})$.
2. **Mean Cooperation Rate:**

$$\rho_C(\pi_i) = \frac{\text{Total Cooperate Decisions Across All Matches}}{M \cdot 200}$$

5.4 Tie-Breaking Order

In the event of an exact tie in average score $\bar{u}_i = \bar{u}_k$:

1. Direct head-to-head match score between π_i and π_k .
2. Total number of match wins.
3. Higher global cooperation rate ρ_C .
4. Earliest submission timestamp.

6 Benchmark Baseline Opponents

All submissions are evaluated against the canonical 9 Axelrod baseline strategies:

Baseline Strategy	Identifier	Decision Rule
Always Cooperate	ALLC	Unconditionally plays $C = 1$ on all rounds.
Always Defect	ALLD	Unconditionally plays $D = 0$ on all rounds.
Tit-for-Tat	TFT	Starts with C , then replicates opponent's previous action.
Tit-for-Two-Tats	TF2T	Defects only after two consecutive opponent defections.
Grim Trigger	GRIM	Cooperates until opponent defects once; permanently defects thereafter.
Win-Stay Lose-Shift	PAVLOV	Repeats action if payoff was R or T ; shifts action if payoff was S or P .
Generous Tit-for-Tat	GTFT	Replicates opponent action, but forgives defection with $q = 1/3$.
Random	RAND	Selects C or D uniformly at random ($p = 0.5$).
Prober (Detective)	PROBER	Plays (C, D, C, C) initially; exploits unprovoked if opponent never retaliates.